

Week 8 Cheat Sheet

Multi-Agent Pathfinding (MAPF)

1. Multi-Agent Pathfinding (MAPF)

MAPF studies how to compute collision-free paths for multiple agents moving in a shared environment.

Goal:

Find trajectories $\Gamma_{1:k}$ for k agents such that:

- Each agent reaches its goal
- No collisions occur between agents
- A global cost function is minimized

2. MAPF Problem Definition

State Space

$$X = \{(x, y)\}$$

Discrete finite grid positions.

Example:

$$(1, 1), (1, 2), (2, 1), (2, 2), \dots$$

Action Space

$$U = \{\text{left, right, up, down, wait}\}$$

Transition Function

$$T : X \times U \rightarrow X$$

Example:

$$T((1, 1), \text{right}) = (2, 1)$$

Collision Constraint

Paths must satisfy:

s.t. no collisions occur between agents

Types of collisions:

- Vertex collision (same cell)
- Edge collision (swap positions)

3. Cost Functions

Two common MAPF objective functions:

Sum of Costs

$$J = \sum_i |\Gamma_i|$$

Total number of steps taken by all agents.

Makespan

$$J = \max_i |\Gamma_i|$$

Time until the last agent reaches its goal.

4. k-Agent A*

Direct extension of A* to multiple agents.

State space size

$$|X|^k$$

Action space size

$$|U|^k$$

Because each agent chooses an action simultaneously.

Properties

- Optimal
- Complete
- Very slow due to exponential growth

5. Prioritized Planning

Assign a priority ordering to agents.

Algorithm:

1. Sort agents by priority
2. Plan path for highest priority agent
3. Treat that path as an obstacle

4. Plan next agent

Properties

- Fast
- Not optimal
- Can cause large delays for low priority agents

6. Operator Decomposition

Instead of expanding the full joint action space:

$$|U|^k$$

expand one agent action at a time.

This reduces branching and improves search efficiency.

7. Independence Detection

Idea: plan independently unless conflicts occur.

Steps:

1. Plan paths for each agent independently
2. Detect conflicts
3. Merge conflicting agents into a group
4. Plan joint path for that group

8. M* Algorithm

Extension of independence detection.

Agents plan independently but dynamically switch to joint planning when conflicts occur.

This reduces unnecessary joint search.

9. Increasing Cost Tree Search (ICTS)

Also called Incremental Tree Cost Search.

Idea: search over combinations of individual path costs.

Define a cost vector:

$$\langle c_1, c_2, \dots, c_k \rangle$$

Example:

$$\langle 3, 3, 1 \rangle$$

If collisions occur, increase costs:

$\langle 4, 3, 1 \rangle$

Then

$\langle 4, 4, 1 \rangle$

Then

$\langle 4, 4, 2 \rangle$

Search over the cost tree using Breadth First Search.

10. Conflict Based Search (CBS)

Two-level search.

High Level

Build a constraint tree.

When a collision occurs:

- Add constraint for agent A
- Add constraint for agent B

Each constraint creates a new branch.

Low Level

Replan the constrained agent using A*.

Properties

- Optimal
- Complete
- Slower than prioritized planning

11. Summary of MAPF Algorithms

Algorithm	Optimal	Speed
k-Agent A*	Yes	Very Slow
Prioritized Planning	No	Fast
Independence Detection	Sometimes	Medium
M*	Yes (in many cases)	Medium
ICTS	Yes	Slow
CBS	Yes	Medium-Slow